

liboqs 라이브러리 설명서

Technical Documentation

김주현

May 28, 2026

Contents

1 개요 (Overview)	3
1.1 프로젝트 목적	3
1.2 코드베이스 스냅샷	3
1.3 지원 알고리즘 범주	3
2 설치 및 빌드 (Build & Install)	3
2.1 현재 확인된 빌드 특성	3
2.2 권장 빌드 절차	3
2.3 산출물 확인	4
2.4 테스트 실행 예시	4
3 라이브러리 구조와 핵심 모듈 (Architecture)	4
3.1 디렉터리 구조	4
3.2 레이어 관점	4
3.3 KEM 디스패치 모델	4
3.4 정적 라이브러리 분리	4
4 공개 API 사용 가이드 (Public API)	5
4.1 핵심 헤더	5
4.2 라이프사이클 API	5
4.3 KEM API 흐름	5
4.4 메모리 API 권장 패턴	5
4.5 알고리즘 선택 시 주의	5
5 예제 코드와 실행 결과 설명 (Examples)	6
5.1 KEM 기본 사용 예제	6
5.2 실행 결과 해석	6
5.3 테스트 코드 기반 참고	6
6 성능/보안 고려사항 (Performance & Security)	7
6.1 성능 관점	7
6.2 보안 관점	7
6.3 운영 권장사항	7
7 한계점 (Limitations)	7
7.1 코드 스냅샷 기준 관찰된 리스크	7
7.2 문서 범위 한계	7

8	개발 규칙 (Development Rules)	8
8.1	코딩/리뷰 규칙	8
8.2	테스트 규칙	8
8.3	문서 유지보수 규칙	8
8.4	참고 링크	8
9	KAT 실행/판정 가이드 (기초 + 트러블슈팅)	8
9.1	이 장의 목적	8
9.2	기초 개념: 종료코드(Exit Code)	9
9.3	자주 발생하는 에러 해석	9
9.4	현재 프로젝트 기준 KAT 실행 명령	9
9.5	성공 판정 규칙	9
9.6	문서 목차를 이렇게 나눈 이유	10

1. 개요 (Overview)

1.1. 프로젝트 목적

liboqs-analysis는 포스트양자암호(PQC) 라이브러리 liboqs의 분석/실험 목적 변형 빌드로 보이며, 현재 코드베이스 기준으로는 KEM(Key Encapsulation Mechanism) 경로에 집중된 구성이다.

핵심 퍼블릭 헤더는 include/oqs/oqs.h, include/oqs/kem.h, include/oqs/common.h이며, 산출물은 build/lib/liboqs-algorithms.a, build/lib/liboqs-internal.a 정적 라이브러리 형태로 확인된다.

ABCDDFDHJHFKJSHHKJHKJHJ

1.2. 코드베이스 스냅샷

- 공개 인터페이스: include/oqs/*.h
- 핵심 구현: src/common, src/kem
- 테스트: tests/kat_kem.c, tests/kat_sig.c, tests/test_helpers.*

1.3. 지원 알고리즘 범주

범주	예시 알고리즘	근거 파일
KEM	HQC-128/192/256, ML-KEM-512/768/1024	include/oqs/kem.h, src/kem/kem.c
Signature	ML-DSA, SLH-DSA 등 식별자 다수	include/oqs/sig.h
Common Crypto	AES, SHA2, SHA3, RNG	src/common/*, include/oqs/common.h

Assumption

본 문서는 사용자 제공 작업 디렉터리(build/lib)와 상위 프로젝트 트리 스캔 결과를 기반으로 작성되었다. 상위 원본 저장소(예: upstream liboqs)와의 동기화 상태는 본 문서 범위에 포함하지 않는다.

2. 설치 및 빌드 (Build & Install)

2.1. 현재 확인된 빌드 특성

include/oqs/oqsconfig.h 기준:

- 버전: OQS_VERSION_TEXT = 0.15.0
- 플랫폼: x86_64 Linux, 배포 빌드 플래그 활성화
- OpenSSL 연동: OQS_USE_OPENSSL=1
- 스레드: OQS_USE_PTHREADS=1

2.2. 권장 빌드 절차

프로젝트 루트에서 다음 흐름을 권장한다.

Listing 1: Configure + Build 예시 (CMake)

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
2 cmake --build build -j
```

2.3. 산출물 확인

Listing 2: 주요 정적 라이브러리 산출물

```
1 build/lib/liboqs-algorithms.a
2 build/lib/liboqs-internal.a
```

2.4. 테스트 실행 예시

Listing 3: KAT 기반 KEM 테스트 예시

```
1 # Binary path may differ by build configuration
2 ./build/tests/kat_kem ML-KEM-768
```

Assumption

저장소 루트에 CMakeLists.txt가 존재한다고 가정하고 표준 CMake 명령을 제시했다. 실제 빌드 시스템 차이가 있으면 동일 원칙(구성-빌드-테스트)으로 명령을 치환한다.

3. 라이브러리 구조와 핵심 모듈 (Architecture)

3.1. 디렉터리 구조

Listing 4: 핵심 디렉터리 개요

```
1 include/oqs/      # public API headers
2 src/common/       # common crypto utilities (AES/SHA/RNG/memory)
3 src/kem/          # KEM interface and algorithm implementations
4 tests/            # KAT and helper-based validation code
5 build/lib/        # static library outputs
```

3.2. 레이어 관점

- API Layer: include/oqs/*.h
- Dispatch Layer: src/kem/kem.c (알고리즘 식별자 매핑, OQS_KEM_new)
- Algorithm Layer: src/kem/hqc/*, src/kem/ml_kem/*
- Utility Layer: src/common/* (메모리/난수/해시/대칭키)

3.3. KEM 디스패치 모델

OQS_KEM_new(method_name)가 문자열 식별자를 기반으로 알고리즘 인스턴스를 생성한다. 호출자는 생성된 OQS_KEM 구조체의 길이 필드와 함수 포인터를 통해 keypair, encaps, decaps를 공통 방식으로 호출한다.

3.4. 정적 라이브러리 분리

라이브러리	역할(추정)	관련 모듈
liboqs-algorithms.a	알고리즘 구현 본체	KEM/SIG 알고리즘 단위 코드
liboqs-internal.a	공통 내부 유틸리티	SHA3 저수준, AES, RNG 등

Assumption

정적 라이브러리별 상세 오브젝트 매핑은 빌드 스크립트 전체를 읽지 않고 현재 트리 및 build/build.txt 단서를 기준으로 기술했다.

4. 공개 API 사용 가이드 (Public API)

4.1. 핵심 헤더

- <oqs/oqs.h>: 통합 진입 헤더
- <oqs/common.h>: 상태 코드, 초기화, 메모리, CPU 확장 체크
- <oqs/kem.h>: KEM 객체/함수
- <oqs/rand.h>: RNG 선택 및 바이트 생성

4.2. 라이프사이클 API

함수	설명
OQS_init()	라이브러리 전역 초기화(OpenSSL/CPU feature 준비 포함)
OQS_destroy()	전역 자원 정리
OQS_thread_stop()	멀티스레드 환경에서 스레드 종료 전 정리
OQS_version()	라이브러리 버전 문자열 반환

4.3. KEM API 흐름

1. OQS_KEM_alg_identifier(i) 또는 문자열 상수로 알고리즘 선택
2. OQS_KEM_new(name)로 객체 생성
3. 길이 필드(length_public_key 등) 기반 버퍼 할당
4. OQS_KEM_keypair, OQS_KEM_encaps, OQS_KEM_decaps 호출
5. OQS_KEM_free() 및 민감정보 secure free

4.4. 메모리 API 권장 패턴

- 민감정보(비밀키/공유비밀): OQS_MEM_secure_free
- 비민감정보(공개키/암호문): OQS_MEM_insecure_free
- 비교는 memcmp 대신 OQS_MEM_secure_bcmp 우선 검토

4.5. 알고리즘 선택 시 주의

OQS_KEM_alg_count()와 OQS_KEM_alg_identifier()로 열거되는 목록이 실행 시점의 enabled 상태와 동일하지 않을 수 있으므로, 실사용 시 OQS_KEM_alg_is_enabled() 체크를 병행한다.

Assumption

include/oqs/kem.h의 OQS_KEM_algs_length와 src/kem/kem.c의 식별자 배열 길이가 동일 전제를 가져야 안전하다. 코드 스냅샷상 두 값의 불일치 가능성이 보여 운영 전 점검이 필요하다.

5. 예제 코드와 실행 결과 설명 (Examples)

5.1. KEM 기본 사용 예제

Listing 5: ML-KEM-768 encaps/decaps 최소 예제

```
1 #include <oqs/oqs.h>
2 #include <oqs/kem.h>
3 #include <oqs/common.h>
4
5 int main(void) {
6     OQS_init();
7
8     OQS_KEM *kem = OQS_KEM_new(OQS_KEM_alg_ml_kem_768);
9     if (kem == NULL) return 1;
10
11     uint8_t *pk = OQS_MEM_malloc(kem->length_public_key);
12     uint8_t *sk = OQS_MEM_malloc(kem->length_secret_key);
13     uint8_t *ct = OQS_MEM_malloc(kem->length_ciphertext);
14     uint8_t *ss_e = OQS_MEM_malloc(kem->length_shared_secret);
15     uint8_t *ss_d = OQS_MEM_malloc(kem->length_shared_secret);
16
17     if (OQS_KEM_keypair(kem, pk, sk) != OQS_SUCCESS) return 2;
18     if (OQS_KEM_encaps(kem, ct, ss_e, pk) != OQS_SUCCESS) return 3;
19     if (OQS_KEM_decaps(kem, ss_d, ct, sk) != OQS_SUCCESS) return 4;
20
21     int ok = (OQS_MEM_secure_bcmp(ss_e, ss_d, kem->length_shared_secret
22                                     ) == 0);
23
24     OQS_MEM_secure_free(sk, kem->length_secret_key);
25     OQS_MEM_secure_free(ss_e, kem->length_shared_secret);
26     OQS_MEM_secure_free(ss_d, kem->length_shared_secret);
27     OQS_MEM_insecure_free(pk);
28     OQS_MEM_insecure_free(ct);
29     OQS_KEM_free(kem);
30     OQS_destroy();
31
32     return ok ? 0 : 5;
33 }
```

5.2. 실행 결과 해석

정상 흐름에서는 keypair -> encaps -> decaps가 모두 OQS_SUCCESS를 반환하고, ss_e와 ss_d가 동일해야 한다.

5.3. 테스트 코드 기반 참고

tests/kat_kem.c는 NIST KAT 스타일 출력 형식(count, seed, pk, sk, ct, ss)을 생성하며, 다중 카운트 검증은 -all 플래그로 수행한다.

Assumption

예제 컴파일 명령은 프로젝트의 실제 export target 설정에 따라 달라질 수 있으므로, 헤더/라이브리리 경로는 환경에 맞춰 조정한다.

6. 성능/보안 고려사항 (Performance & Security)

6.1. 성능 관점

- 빌드 플래그: -O3, -fdata-sections, -ffunction-sections, -gc-sections
- 아키텍처 최적화 분기: oqsconfig.h의 x86_64 관련 define, AVX 계열 조건부 사용
- 알고리즘별 trade-off: ML-KEM/HQC 파라미터 세트별 키/암호문 크기 및 연산비용 상이

6.2. 보안 관점

- 민감감 버퍼는 OQS_MEM_cleanse, OQS_MEM_secure_free 경로 사용
- 공유비밀 비교는 타이밍 누설 완화를 위해 OQS_MEM_secure_bcmp 사용
- RNG 소스 변경 시(OQS_randombytes_switch_algorithm) 테스트/검증 절차 필수
- 오류 처리 시 초기 반환보다 정리(cleanup) 블록 일원화 권장

6.3. 운영 권장사항

1. 알고리즘 enable 여부를 시작 시점에 검사해 fallback 정책을 정의한다.
2. OpenSSL/플랫폼 업데이트 시 회귀 테스트(KAT + 상호운용성)를 수행한다.
3. CI에 최소 1개 KEM과 1개 SIG smoke test를 배치한다.

7. 한계점 (Limitations)

7.1. 코드 스냅샷 기준 관찰된 리스크

- include/oqs/kem.h의 OQS_KEM_algs_length(41) 대비 src/kem/kem.c의 식별자 배열 항목 수가 축소되어 보인다.
- src/kem/kem.c의 일부 분기에서 fallback return 문이 누락된 형태로 보여, 컴파일러/설정에서 경고 또는 UB 가능성이 있다.
- build/build.txt는 공식 빌드 매니페스트보다 분석 메모 성격이 강해 재현 가능한 빌드 정의로 사용하기 어렵다.

7.2. 문서 범위 한계

본 문서는 현재 로컬 트리 탐색 결과 기반이며, 외부 종속성 버전(OpenSSL 세부 ABI 등)과 상위 저장소 정책 변화는 자동 반영되지 않는다.

Assumption

상기 리스크는 현재 체크아웃 상태를 기준으로 하며, 다른 브랜치/커밋에서는 이미 해소되었을 수 있다.

8. 개발 규칙 (Development Rules)

8.1. 코딩/리뷰 규칙

1. 공개 API 변경 시 `include/oqs/*.h`와 구현체를 동시에 갱신한다.
2. 알고리즘 추가/삭제 시 식별자 목록, `count`, `enable` 분기를 일관되게 수정한다.
3. 민감 데이터 메모리 해제는 `secure API`를 우선 적용한다.
4. 실패 경로는 단일 `cleanup` 라벨로 정리해 누수/중복 해제를 방지한다.

8.2. 테스트 규칙

- KEM: `keypair/encaps/decaps` 왕복 성공 + 공유비밀 일치 필수
- SIG: 서명/검증 정상 케이스 + 변조 입력(`bitflip`) 실패 케이스 필수
- 알고리즘 `enable/disable` 설정별 smoke test 수행

8.3. 문서 유지보수 규칙

- 새 알고리즘 추가 시 본 문서 3장(구조), 4장(API), 5장(예제) 동시 갱신
- 빌드 옵션 변경 시 2장 업데이트
- Assumption 항목은 실제 확인 시 제거하거나 근거 링크로 대체

8.4. 참고 링크

- 프로젝트 루트: `/home/wngus/projects/liboqs-analysis`
- Public headers: `include/oqs/`
- Test entry points: `tests/kat_kem.c`, `tests/kat_sig.c`

9. KAT 실행/판정 가이드 (기초 + 트러블슈팅)

9.1. 이 장의 목적

이 장은 실제 작업 중 가장 자주 헛갈리는 항목을 빠르게 해소하기 위한 운영 가이드다. 특히 다음 질문에 답한다.

- 왜 종료코드 0이면 성공인가?
- 127은 무엇을 의미하는가?
- `kat_kem`, `kat_sig` 실행 파일이 없다는 에러는 왜 발생하는가?
- 어떤 명령으로 현재 구성의 KAT 성공 여부를 한 번에 확인하는가?

9.2. 기초 개념: 종료코드(Exit Code)

유닉스/리눅스 관례에서 프로그램 종료 상태는 정수 코드로 표현된다.

- 0: 정상 종료(성공)
- 0 이외: 비정상 종료(실패/오류)

셸(zsh, bash)에서는 직전 명령의 종료코드를 \$?로 확인한다.

Listing 6: 종료코드 확인 예시

```
1 ./build/tests/kat_kem ml-kem-512 > /tmp/kat_mlkem512.out
2 echo $?
```

9.3. 자주 발생하는 에러 해석

1) zsh: no such file or directory: ./build/tests/kat_sig

의미: 알고리즘 실패가 아니라, 실행 파일 자체가 아직 생성되지 않았다는 뜻이다.

해결:

```
1 make kat_sig
```

2) 종료코드 127

의미: 명령 실행 실패(대부분 실행 파일 경로 오류/파일 부재). 즉 kat_sig ML-DSA-44 => 127은 “ML-DSA가 틀림”이 아니라 “kat_sig 파일이 없음”에 가깝다.

9.4. 현재 프로젝트 기준 KAT 실행 명령

KEM 일괄 실행

```
1 for alg in hqc-128 hqc-192 hqc-256 ml-kem-512 ml-kem-768 ml-kem-1024;
  do
2   ./build/tests/kat_kem "$alg" > /tmp/kat_kem_$alg.out
3   echo "kat_kem $alg => $?"
4 done
```

SIG 일괄 실행

```
1 for alg in ML-DSA-44 ML-DSA-65 ML-DSA-87 SLH_DSA_PURE_SHA2_128S; do
2   ./build/tests/kat_sig "$alg" > /tmp/kat_sig_$alg.out
3   echo "kat_sig $alg => $?"
4 done
```

9.5. 성공 판정 규칙

실무에서는 아래 규칙만 기억하면 된다.

- 각 알고리즘 줄의 결과가 => 0이면 해당 KAT 성공
- => 0이 아니면 실패 로그를 /tmp/kat_*.out에서 확인
- 127이면 먼저 바이너리 존재 여부(make kat_kem, make kat_sig)를 점검

9.6. 문서 목차를 이렇게 나눈 이유

본 문서는 학습 순서와 디버깅 효율을 기준으로 구성했다.

1. **개요/구조 먼저**: 프로젝트 목적과 파일 위치를 알아야 경로 오류를 줄일 수 있다.
2. **빌드/실행 다음**: 실제 산출물(`kat_kem`, `kat_sig`) 생성이 가장 먼저 필요하다.
3. **판정 규칙 별도 분리**: 종료코드 해석을 독립시켜 오해(127 vs 알고리즘 실패)를 방지한다.
4. **트러블슈팅 독립 장**: 에러 상황에서 바로 검색해 해결할 수 있도록 했다.
5. **제약/확장 마지막**: 현재 상태를 검증한 뒤 제한사항을 보는 흐름이 이해에 유리하다.

Assumption

본 가이드는 현재 `liboqs-analysis` 작업 트리의 커스텀 빌드/테스트 구성을 기준으로 한다. 상위 원본 저장소와 1:1 동등 동작을 보장하는 문서는 아니다.